

torch.nn.functional.interpolate#

[https://pytorch.org/docs/stable/generated/torch.nn.functional.interpolate.ht](https://pytorch.org/docs/stable/generated/torch.nn.functional.interpolate.html)

Description:

Down/up samples the input. Tensor interpolated to either the given size or the given `scale_factor`. The algorithm used for interpolation is determined by `mode`. Currently temporal, spatial and volumetric sampling are supported, i.e. expected inputs are 3-D, 4-D or 5-D in shape. The input dimensions are interpreted in the form: mini-batch x channels x [optional depth] x [optional height] x width.

Function Signature

```
torch.nn.functional.interpolate(input, size=None,
                                scale_factor=None, mode='nearest', align_corners=None,
                                recompute_scale_factor=None, antialias=False)[source]#
```

Parameters

Return type

Returns:

Tensor

Notes & Warnings

NOTE

Note With mode='bicubic', it's possible to cause overshoot. For some dtypes, it can produce negative values or values greater than 255 for images. Explicitly call `result.clamp(min=0,max=255)` if you want to reduce the overshoot when displaying the image. For uint8 inputs, it already performs saturating cast operation. So, no manual clamp operation is needed.

NOTE

Note Mode mode='nearest-exact' matches Scikit-Image and PIL nearest neighbours interpolation algorithms and fixes known issues with mode='nearest'. This mode is introduced to keep backward compatibility. Mode mode='nearest' matches buggy OpenCV's INTER_NEAREST interpolation algorithm.

NOTE

Note The gradients for the dtype float16 on CUDA may be inaccurate in the upsample operation when using modes ['linear', 'bilinear', 'bicubic', 'trilinear', 'area']. For more details, please refer to the discussion in [issue#104157](#).

NOTE

Note This operation may produce nondeterministic gradients when given tensors on a CUDA device. See [Reproducibility](#) for more information.

Detailed Documentation

Documentation

Down/up samples the input.

Tensor interpolated to either the given size or the given `scale_factor`

The algorithm used for interpolation is determined by `mode`.

Currently temporal, spatial and volumetric sampling are supported, i.e. expected inputs are 3-D, 4-D or 5-D in shape.

The input dimensions are interpreted in the form: mini-batch x channels x [optional depth] x [optional height] x width.

The modes available for resizing are: nearest, linear (3D-only), bilinear, bicubic (4D-only), trilinear (5D-only), area, nearest-exact

`input` (Tensor) – the input tensor

`size` (int or Tuple[int] or Tuple[int, int] or Tuple[int, int, int]) – output spatial size.

`scale_factor` (float or Tuple[float]) – multiplier for spatial size. If `scale_factor` is a tuple, its length has to match the number of spatial dimensions; `input.dim() - 2`.

`mode` (str) – algorithm used for upsampling: 'nearest' | 'linear' | 'bilinear' | 'bicubic' | 'trilinear' | 'area' | 'nearest-exact'. Default: 'nearest'

`align_corners` (bool, optional) – Geometrically, we consider the pixels of the input and output as squares rather than points. If set to True, the input and output tensors are

aligned by the center points of their corner pixels, preserving the values at the corner pixels. If set to False, the input and output tensors are aligned by the corner points of their corner pixels, and the interpolation uses edge value padding for out-of-boundary values, making this operation independent of input size when `scale_factor` is kept the same. This only has an effect when mode is 'linear', 'bilinear', 'bicubic' or 'trilinear'. Default: False

`recompute_scale_factor` (bool, optional) – recompute the `scale_factor` for use in the interpolation calculation. If `recompute_scale_factor` is True, then `scale_factor` must be passed in and `scale_factor` is used to compute the output size. The computed output size will be used to infer new scales for the interpolation. Note that when `scale_factor` is floating-point, it may differ from the recomputed `scale_factor` due to rounding and precision issues. If `recompute_scale_factor` is False, then size or `scale_factor` will be used directly for interpolation. Default: None.

`antialias` (bool, optional) – flag to apply anti-aliasing. Default: False. Using anti-alias option together with `align_corners=False`, interpolation result would match Pillow result for downsampling operation. Supported modes: 'bilinear', 'bicubic'.

With mode='bicubic', it's possible to cause overshoot. For some dtypes, it can produce negative values or values greater than 255 for images. Explicitly call `result.clamp(min=0,max=255)` if you want to reduce the overshoot when displaying the image. For uint8 inputs, it already performs saturating cast operation. So, no manual clamp operation is needed.

Mode `mode='nearest-exact'` matches Scikit-Image and PIL nearest neighbours interpolation algorithms and fixes known issues with mode='nearest'. This mode is introduced to keep backward compatibility. Mode `mode='nearest'` matches buggy OpenCV's `INTER_NEAREST` interpolation algorithm.

The gradients for the dtype float16 on CUDA may be inaccurate in the upsample

operation when using modes ['linear', 'bilinear', 'bicubic', 'trilinear', 'area']. For more details, please refer to the discussion in [issue#104157](#).

This operation may produce nondeterministic gradients when given tensors on a CUDA device. See [Reproducibility](#) for more information.